



Programação Dinâmica: Problema do Subconjunto Soma

Universidade Federal de Sergipe (UFS)
Programa de Pós Graduação em Ciência da Computação (PROCC)
Disciplina: Projeto e Análise de Algoritmos
Docente: Prof. Dr. Leonardo Nogueira Matos
Discente: Bruna dos Santos Silva



Sumário

- Contextualização;
 - Exemplo Simples,
 - Exemplo Prático.
- Classificação do problema;
- Técnica utilizada;
- Demonstração do algoritmo;
- Referências.



Problema: Subconjunto Soma (Subset Sum)

- Definição formal:

Dado um conjunto de números inteiros positivos $S=\{s_1,s_2,\dots,s_n\}$ e um valor alvo T , determinar se existe algum subconjunto de S cuja soma é exatamente igual a T .

- Exemplo simples:

- Entrada:

- $S = \{3, 34, 4, 12, 5, 2\}$

- $T = 9$

- Saída: True, $\{4,5\}$ soma 9



Problema: Subconjunto Soma (Subset Sum)

- Exemplo II: Conexão com Análise de Sentimentos em projetos *open source*
 - Para cada comentário em um pull request (PR) podemos atrelar um valor ao sentimento: Positivo (1), Neutro (0) e Negativo (-1);
 - $PR = \{-1, 0, 1, -1, -1\}$
 - O valor alvo T desejado é o equilíbrio nos sentimentos = 0
 - No pull request do exemplo, conseguimos um subconjunto $S(PR)$ em que exista o valor T ?
 - Podemos classificar e relacionar os tipos de pull requests com a meta de equilíbrio nos sentimentos:
 - PRs de melhorias podem ser mais neutros que PRs de bugs?



Classificação: Subconjunto Soma (Subset Sum)

- NP-completude é uma forma de classificar problemas computacionais de acordo com a sua dificuldade e com o tipo de solução possível. Um problema é P se pode ser resolvido rapidamente por um algoritmo determinístico. Um problema é NP se, dada uma solução candidata, seja possível verificar se ela é válida, mesmo que não tenha como encontrar essa solução rapidamente. Dentro da classe NP, existe o tipo NP-Completo e NP-Difícil.
- O **Subset Sum** é classificado como **NP-Completo**, pois:
 - Está em NP,
 - É tão difícil quanto qualquer problema de NP.



Técnica: Subconjunto Soma (Subset Sum)

- É possível verificar todas as combinações possíveis com recursão.
- Complexidade: $O(2^n)$

```
def subset_sum(nums, target, i=0):  
    if target == 0:  
        return True  
  
    if i >= len(nums):  
        return False  
  
    if target < 0:  
        return False  
  
    return subset_sum(nums, target - nums[i], i + 1)  
        or subset_sum(nums, target, i + 1)
```

Técnica: Subconjunto Soma (Subset Sum)

- **Programação Dinâmica:**

- Guarda em uma tabela as soluções das sub-instâncias da instância original.

SUBSET-SUM-PROG-DIN (p, n, c)

1 para i crescendo de 0 até n

2 $t[i, 0] := 1$

3 para j crescendo de 1 até c

4 $t[0, j] := 0$

5 para i crescendo de 1 até n

6 $b := t[i-1, j]$

7 se $b = 0$ e $p[i] \leq j$

8 $b := t[i-1, j-p[i]]$

9 $t[i, j] := b$

10 devolva $t[n, c]$

Dúvidas?

Obrigada pela atenção.

- Github: https://github.com/BrunnaSilva/PAA_Seminario_NP

Contatos:

- E-mail: bruna.silva@dcomp.ufs.com
- LinkedIn: <https://www.linkedin.com/in/ibruna/>





Referências

- CORMEN, Thomas H. et al. ***Introduction to Algorithms***. 3. ed. Cambridge, MA: MIT Press, 2009. ISBN 978-0-262-03384-8. Pág.: 1128-1134
- FEÓFILOFF, Paulo. ***Como encontrar um subconjunto com uma dada soma (Subset Sum)***. São Paulo: Instituto de Matemática e Estatística, Universidade de São Paulo, 02 nov. 2020. Disponível em: https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/mochila-subsetsum.html. Acesso em: 20 nov. 2025.